

Hierarchical Agglomerative Clustering and LSTM-based Load Prediction for Dynamic Spectrum Allocation

Lei Liu, Hamed Mosavat-Jahromi, Lin Cai
Dept. Electrical and Computer Engineering
University of Victoria
 Victoria, Canada

Email: heu_liulei@hotmail.com, hamedmosavat@uvic.ca, cai@ece.uvic.ca

David Kidston
Communications Research Centre Canada
 Ottawa, Canada
 Email: david.kidston@canada.ca

Abstract—To improve spectrum efficiency without interfering with licensed users, reliable prediction of spectrum occupancy plays a pivotal role in a dynamic spectrum allocation (DSA) system. A reliable machine learning method capable of exploring the long-term correlation in the data is using a neural network with long short-term memory (LSTM). However, in the situation that there are multiple sensors in the network, how to effectively exploit the spatial correlation among these sensors' data for accurate spectrum prediction remains an open issue. Directly applying LSTM to multiple series may even reduce the prediction accuracy if some series are uncorrelated. In this article, we propose a method of clustering to aid in predicting multi-dimensional received power based on the hierarchical agglomerative clustering (HAC) model, which clusters the correlated series with high spearman's rank correlation coefficient (SRCC). Similar series are grouped into clusters and trained to predict independently using HAC. By ensuring uncorrelated series do not influence each other, the LSTM prediction accuracy is improved. A low-pass filter is used to remove high-frequency noise components and further reduce the prediction error. Experimental results show that our method significantly increases the prediction accuracy in all cases.

Index Terms—Dynamic spectrum allocation (DSA), long short-term memory (LSTM), spectrum prediction, hierarchical agglomerative clustering (HAC)

I. INTRODUCTION

Radio spectrum is an important part of wireless communications. How radio spectrum is allocated to users continues to receive the attention of scholars. With the traditional static spectrum allocation, spectrum resources are allocated to licensees for a long period of time. Although simple, it is not flexible to meet the dynamic requirements of ever-changing networks. This results in low spectral efficiency.

Dynamic spectrum allocation (DSA) has been proposed [1] to improve spectral efficiency by having each spectrum band dynamically allocated to a licensee over a short time period. For DSA to work effectively, temporal spectrum usage prediction is critical. Spectrum usage can be predicted by analyzing the received signal power in the time domain. In this way, the dynamic spectrum allocator can predict the spectrum occupancy of the selected location in the following time slots, i.e., whether the channel will be occupied with

a high probability or not. Accurate spectrum prediction can simplify the sensing process and improve sensing efficiency.

Spectrum sensing data collected by multiple sensors usually have multi-dimensional characteristics, and multi-dimensional data analysis can be carried out by using appropriate time series analysis methods. Some traditional methods of spectrum sensing data analysis are autoregressive integrated moving average (ARIMA) [2] and generalized autoregressive conditional heteroskedasticity (GARCH) [3] models, but these methods are difficult to deal with high dimensional problems. As a classic deep learning method of time series prediction, recurrent neural networks (RNNs) show good performance in many applications [4], but it suffers from vanishing gradient problem. To improve RNNs, adding long short-term memory (LSTM) solves the problem of vanishing gradient [5].

Different from RNNs, a typical LSTM cell has four gates, which have their own weights and biases. By using the four special gates, LSTM networks have the ability to learn long-term correlations in time series and are able to handle the problem of complex multi-dimensional time series prediction. In recent years, LSTM has been used to analyze multiple time series data in many applications including translation modeling [6] and traffic classification [7], and has shown excellent performance for time series prediction in spectrum prediction at a single location [8]. Building a separate LSTM model for each location inevitably increases complexity. On the other hand, for multi-series prediction, uncorrelated series can reduce the prediction accuracy and will lead to complicated prediction models. Furthermore, random noise in each series will negatively affect the learning performance. Directly using LSTM for multi-dimensional time series prediction has failed to obtain satisfactory prediction results [9].

Clustering is one of the unsupervised machine learning methods. In clustering, according to a certain standard (such as distance), a dataset is divided into classes or clusters so that correlated data can be gathered together. In wireless communication networks, the channel occupancy sensed by nearby sensors will have a high correlation given the spatial correlation property of wireless signals propagation. As one of

the most popular clustering methods, hierarchical agglomerative clustering (HAC) aggregates the data categories according to the smallest distance and iterates through the process, until the number of categories reaches the expected value or other termination conditions are met, making it suitable for finding the most similar series in a large amount of spectrum sensing data, which can be used to improve prediction performance. To minimize the impact of uncorrelated time series and improve the prediction accuracy, we propose the novel approach of using correlated multi-series sensor data and clustering them to predict the received power at each location.

The contributions of this article are the following.

- The Spearman's rank correlation coefficient (SRCC)-based HAC method is used to cluster monotonically similar series together.
- A prediction method with the proposed HAC and LSTM models (HAC-LSTM) is developed to improve multiple time series prediction by removing uncorrelated series.
- A data pre-processing method based on a low-pass filter is designed to remove high-frequency noise components and improve prediction accuracy.
- Real spectrum sensing data are used to validate the proposed approach and demonstrate its superior performance.

The following outlines the rest of this article. Section II introduces the related works on time series prediction methods. Section III describes the sensor network architecture and spectrum sensing data. Section IV presents the proposed prediction method combining HAC and LSTM. Section V shows the performance evaluation. Section VI summarizes this article and further research issues.

II. RELATED WORK

Time series forecasting has recently received extensive attention, especially in industry, for various practical issues, such as server performance estimation, pricing prediction, and spectrum occupancy prediction. Predicting the received power on the channel for future time slots can improve the spectrum resource allocation performance. There are many studies focusing on time series analysis.

ARIMA is a well-known traditional method, using linear model in time series forecasting, which combines autoregressive (AR) and moving average (MA) models together for time series prediction. In [2], ARIMA is used to analyze the spectrum occupancy of the global system for mobile communications (GSM) band. Holt-winters is another traditional time series analysis and forecasting method applicable to non-stationary series containing linear trends and periodic fluctuations, which can be used to analyze periodic time series, such as hourly call arrival rate prediction in cognitive radio (CR) cellular networks [10]. The exponential moving average (EMA) method is used to continuously adapt the model parameters to the changes of non-stationary series and to make short-term forecasts of future trends [11].

Machine learning also performs well in time series analysis. Compared with other time series prediction algorithms,

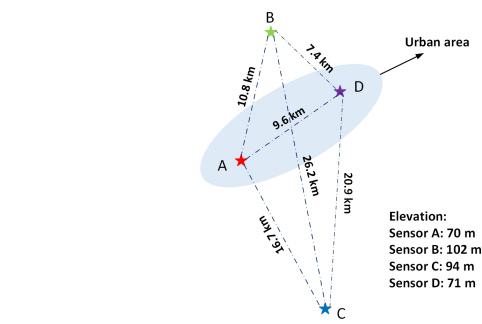


Fig. 1: Topology of the sensor network.

tree-based methods have strong decision-making and analysis capabilities. The tree-based methods do not require the assumption of a normal statistical distribution and have higher prediction accuracy and robustness. Random forest (RF) [12] and gradient boosting decision trees (GBDT) [13] are two typical methods in the tree-based methods.

LSTM as a form of deep neural networks (DNNs) was designed to overcome the vanishing gradient problem of RNNs, providing a model to store long-term information [5] and has shown excellent performance in spectrum prediction [8]. Prophet [14] is a method for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects.

In recent years, many studies have combined different deep learning models to solve more complex spatio-temporal prediction problems. The convolutional LSTM (ConvLSTM) The convolutional LSTM (ConvLSTM) combines convolution and LSTM and has shown good performance in spectrum prediction for DSA [15]. In [16], a spatio-temporal recurrent convolutional network, combining convolutional neural network (CNN) and LSTM, was used for traffic prediction. The spatial dependencies of network-wide traffic can be captured by CNNs, and the temporal dynamics can be learned by LSTM. The above spatio-temporal prediction methods inspired us to combine the spatial and temporal correlations of spectrum sensing data to achieve more accurate predictions.

III. SYSTEM MODEL

Spectrum sensing datasets were collected by four sensors in different locations by a spectrum environment awareness system deployed by Communications Research Centre (CRC) in Ottawa, Canada [17]. The measurements were taken from November 1, 2018 to November 30, 2018 by one B200 USRP Ettus research radio frequency front-end. Fig. 1 shows the topology of the four sensors network. The distance between sensors B and C was the longest (26.2km), and the distance between sensors B and D was the shortest (7.4km). Sensors A and D were deployed in urban areas (shaded part in Fig. 1). The elevations of sensors A, B, C, and D were 70, 102, 94, and 71 meters, respectively.

The system includes a network of sensors with high resolution in time and frequency for the measurements, which

are reported to a cloud-based platform for data processing and analysis. The frequency range spanned from 70 MHz to 6 GHz. The exploratory analysis in this work focuses on a subset of these measurements spanning the 450 MHz and 800 MHz Land Mobile Radio (LMR) bands. The measurements are channel and time-specific. The resolution bandwidth is 15 kHz and the channels were scanned every 1-2 seconds. The scanned results are received powers.

In this article, the data are collected from a channel with an operating frequency of 470.31 MHz for analysis. This channel is used for digital TV, which is almost always available and can be used as a secondary reference signal for calibration and analysis.

Unfortunately, in the datasets, the sensors are not perfectly synchronized, and the sensing intervals are different from time to time. In order to address this problem, a fixed-duration time slot was used to process the datasets and synchronize them. In these datasets, we consider a one-minute slot duration and calculate the average received power per slot for each sensor.

We selected a continuous 10000 minutes (roughly a week) of average received powers for analysis. The last 3500 minutes were separated as the test set to evaluate the performance of prediction (Section V), and the rest were the training set.

IV. PREDICTION METHOD COMBINING HAC AND LSTM (HAC-LSTM)

A. Spearman's rank correlation coefficient (SRCC)

SRCC can be used to measure the correlation between two ordinal series. It measures how ordinal series changes in one dataset as the other ordinal changes. It uses monotone equations to evaluate the correlation of two ordinal series, and is a non-parametric indicator that measures the dependence of two series.

Suppose we have two series x_i and y_i , the formula of SRCC is

$$r_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}, \quad (1)$$

where d_i is the rank difference between each x_i , y_i data pair, and n is the number of data pairs.

SRCC can be applied to both continuous and discrete data. The SRCC, r_s , can take values from +1 to -1. A value of +1 indicates a perfect association of ranks, while a value of zero indicates no association between ranks and -1 indicates a perfect negative association of ranks. The closer r_s is to zero, the weaker the association between the ranks is.

B. Hierarchical agglomerative clustering (HAC)

HAC is the most common type of hierarchical clustering models used to group objects in clusters based on their similarity. The model starts by treating each object as a singleton cluster and then calculates the distance between different clusters. The smaller the distance, the higher the similarity. Next, pairs of similar clusters are successively merged until all clusters have been merged into one big cluster containing all objects. The result is a tree-based representation of the

objects, called a dendrogram. In our work, $1 - |r_s|$ represents the distance. Thus the closer the absolute value of r_s to 1 is, the closer the two series are.

There are many cluster agglomeration methods (i.e., linkage methods) that define the distance between pairs of clusters in their own way. The common linkage methods of HAC models are single linkage, complete linkage, and average linkage. In our work, we use complete linkage to calculate the distance. In complete linkage, the distance between two clusters is defined as the maximum value of all pairwise distances between the elements in cluster A and the elements in cluster B. Therefore, it can be ensured that the series in a cluster are all correlated. It tends to produce more compact clusters. The process is described in Algorithm 1.

Algorithm 1 HAC based on SRCC

Require: $x_1, x_2, \dots, x_k$: series set; $Corr(*, *)$: function of SRCC; λ : threshold; n : target number of cluster

```

1: for  $i = 1$  to  $k$  do
2:    $C_i = x_i$ 
3:   for  $j = 1$  to  $k$  do
4:      $r_{i,j} = Corr(x_i, x_j)$ 
5:   for  $i = 1$  to  $k$  do
6:     for  $j = 1$  to  $k$  do
7:        $Q_{i,j} = r_{i,j}$ 
8:        $Q_{j,i} = Q_{i,j}$ 
9: set initial cluster number:  $p = k$ 
10: while  $p > n$  do
11:   find two cluster  $C_{i^*}$  and  $C_{j^*}$  with the biggest absolute value of SRCC;
12:   if  $|Q(i^*, j^*)| < \lambda$  then
13:     break the whole process
14:   else
15:     merge  $C_{i^*}$  and  $C_{j^*}$ :  $C_{i^*} \cup C_{j^*}$ 
16:     for  $j = j^* + 1$  to  $p$  do
17:       renumber the  $C_{i^*}$  to  $C_{j^*}$ 
18:     delete the  $j^{*th}$  row and  $j^{*th}$  column
19:     for  $j = 1$  to  $p - 1$  do
20:       for  $i$  in  $C_{i^*}$  do
21:          $r_{i,j} = Corr(x_i, x_j)$ 
22:          $Q_{i^*,j} = max(r_{i,j})$ 
23:    $p = p - 1$ 

```

C. Long short-term memory (LSTM)

LSTM networks are explicitly designed to handle the long-term dependency problem, which makes it very suitable for time series prediction. In a typical LSTM network, each cell has four gates, interacting in a special way. Fig. 2 illustrates a single LSTM memory cell [5].

For the LSTM model used in this article, the compact forms of the equations for the forward pass of an LSTM unit from [5] are given by

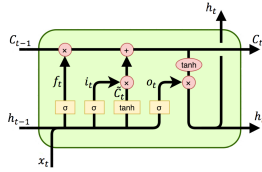


Fig. 2: Long short-term memory cell.

$$\begin{aligned}
 f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f), \\
 i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i), \\
 o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o), \\
 \tilde{c}_t &= \sigma_h(W_c x_t + U_c h_{t-1} + b_c), \\
 c_t &= f_t \circ c_{t-1} + i_t \circ \tilde{c}_t, \\
 h_t &= o_t \circ \sigma_h(c_t),
 \end{aligned} \tag{2}$$

where σ_g and σ_h are the logistic sigmoid function and hyperbolic tangent function, and i, f, o, c , and $\tilde{c}$ are the input gate, forget gate, output gate, cell state, and cell input activation vectors, respectively, all of which are the same size as the hidden state vector h . W, U , and b are the input-output gate weight matrices, hidden-input gate weight matrices, and bias vector parameters, respectively, which need to be learned during training. The subscript t indexes the time step and the operator $\circ$ denotes the Hadamard product.

LSTM uses the approximate gradient calculation algorithm, which updates the weight matrices after each time step. However, the backpropagation algorithm can be used to calculate the entire gradient.

D. Data pre-processing and HAC-LSTM prediction model

Data pre-processing by a low-pass filter: Since high-frequency spectrum usage changes are mainly due to random behaviors that cannot be predicted using time correlation in the data, they will skew the LSTM prediction performance during the training process. In this article, before clustering and prediction, we use a low-pass filter to pre-process the training data to smooth out high-frequency changes.

After data pre-processing, we calculate the SRCC for each pair of series and obtain a correlation coefficient matrix. In clustering, we gather the correlated series whose r_s is closed to $+1$ or -1 together. Finally, we build the LSTM model to predict each cluster independently. The proposed LSTM model includes both LSTM layers and fully connected layers. The LSTM layer is used to get information from input data, and the fully connected layer is used to export the output data, where the inputs are the previous average received powers, and the outputs are the predicted results. The whole system framework is shown in Fig. 3.

V. PERFORMANCE EVALUATION

In this section, we discuss the prediction performance of the proposed HAC-LSTM model. Moreover, the deep learning time series prediction method LSTM [6], the traditional time series analysis method ARIMA [2], the classic machine learning method RF [12], and the popular spatio-temporal

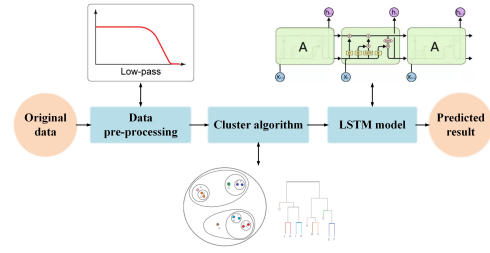


Fig. 3: Prediction method combining HAC and LSTM.

prediction methods CNN-LSTM [16] and ConvLSTM [15] models are used for comparison, according to their reliable and excellent performance in time series forecasting.

A. Clustering result

The network settings and dataset description were introduced in Section III. The four series a, b, c, and d, corresponding to the power received by four sensors A, B, C, and D, respectively. First, we use the SRCC based HAC method to cluster the correlated series. The result is shown in Fig. 4. It can be seen that series b and c are the most correlated, even though the distance between sensors B and C are farthest. This is because, as shown in Fig. 1, sensors A and D are deployed in an urban area. This causes the received power to be substantially affected by the buildings and other obstacles. Sensors B and C are outside the urban area and have relatively high elevation to have a large sensing area overlapping. Thus, it makes sense that the received power collected by sensors B and C is more correlated to each other, but uncorrelated to sensors A and D. In the LSTM model, we used series b and c as the multiple inputs for prediction.

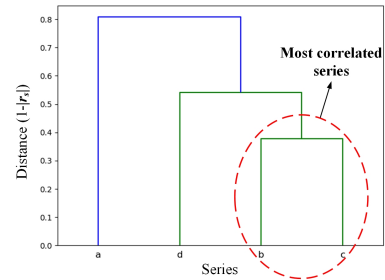


Fig. 4: Clustering result of HAC based on the SRCC.

B. Prediction results without data pre-processing

We designed the HAC-LSTM model for training and predictive analysis. The prediction model is trained according to the organized dataset. 60% of the dataset is allocated to the training phase, and the remaining part is used for the test and generalization. We use the received power of 10 minutes (defined as one slot) to predict the next one. The HAC-LSTM framework is implemented in Python, and the “Keras” library is used to build the LSTM network architecture. Stacked LSTM layers consisting of one hidden layer with 32 units are exploited for the spectrum prediction. The loss function is the

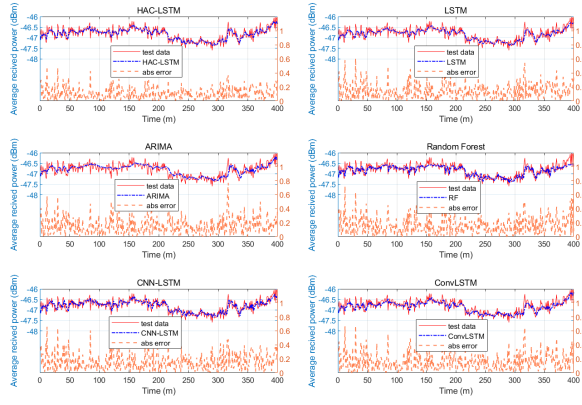


Fig. 5: Prediction results and absolute prediction error.

mean absolute error (MAE). The activation function and the optimization algorithm that are used in the training phase are “tanh” and “adam”, respectively. The learning rate is 0.0005, and the batch size is 64. We put the closely correlated series b and c into our LSTM. After 100 rounds of training, we obtained the prediction results, as shown in Fig. 5.

Fig. 5 shows the prediction results and absolute prediction error for average received power by applying HAC-LSTM, LSTM, ARIMA, RF, CNN-LSTM, and ConvLSTM prediction methods. The “Statsmodels” library in python was used to build the ARIMA architecture. We search the different values of the Bayesian information criterion (BIC) corresponding to different values of p and q for ARIMA [2]. The minimum BIC is achieved when $p = q = 1$, and the difference order $d = 1$. The “Sklearn” library in python was used to build the random forest regressor, and the “GridSearchCV” function was used to automatically select the optimal parameters for the RF prediction method. The CNN-LSTM model contains one one-dimensional CNN layer with 200 filters (filter size is 1×2) and one LSTM layer. The ConvLSTM model contains one ConvLSTM layer with 100 filters (filter size is 1×2) and one LSTM layer. The activation function of 1D CNN layer and ConvLSTM layer is “relu”. The parameters of LSTM layer in 1D CNN-LSTM model and ConvLSTM model are the same as those of LSTM layer in HAC-LSTM model. The learning rate and batch size of CNN-LSTM model and ConvLSTM model are 0.0005 and 64, respectively. The red line, the blue line and the khaki line in Fig. 5 show the real data, the predicted value, and the absolute error of the real data and the predicted value, respectively. As shown in Fig. 5, comparing the prediction results of all methods, the prediction error of HAC-LSTM is lower than the other five methods. However, the prediction errors of all methods are quite high, using the raw measurement data without pre-processing.

C. Prediction results with data pre-processing by a low-pass filter

Next, we used a low-pass filter to remove high-frequency components to increase temporal correlation for the series. We set 0.5 as the filter’s cutoff frequency. This cutoff frequency

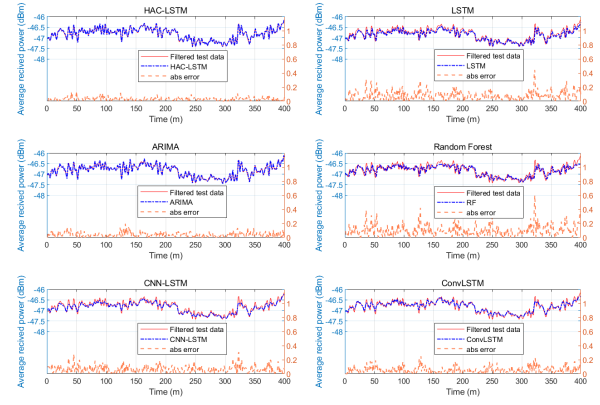


Fig. 6: Prediction results and absolute prediction error with a low-pass filter.

was examined for all datasets. The results show that the cutoff frequency is stable for the channel and is a proper setting for the low-pass filter.

Fig. 6 shows the prediction results by using filtered data. The p, d, q of the ARIMA are 7, 1, and 4, respectively. We can see that compared to Fig. 5, the prediction results by using the filtered data are more accurate for all prediction methods. However, the prediction errors of HAC-LSTM, ARIMA, CNN-LSTM, and ConvLSTM prediction methods are less pronounced than those of LSTM and RF. HAC-LSTM still has the highest prediction accuracy.

TABLE I: Error Comparison of Different Prediction Methods

Prediction Methods	Without low-pass filter					
	HAC-LSTM	LSTM	ARIMA(1,1,1)	RF	CNN-LSTM	ConvLSTM
MAPE	0.2364	0.3043	0.3288	0.3448	0.3373	0.3369
RMSE	0.1410	0.1812	0.1960	0.2055	0.2004	0.2025
Prediction Methods	With low-pass filter					
	HAC-LSTM	LSTM	ARIMA(7,1,4)	RF	CNN-LSTM	ConvLSTM
MAPE	0.0665	0.1785	0.0849	0.2234	0.1461	0.1254
RMSE	0.0390	0.1065	0.0518	0.1339	0.0861	0.0755

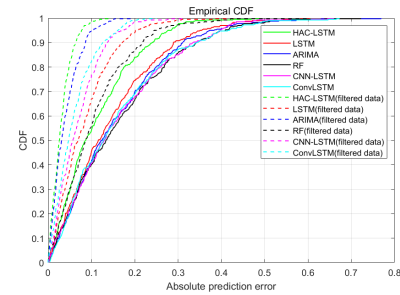


Fig. 7: CDF of the absolute prediction error.

The cumulative distribution functions (CDF) of the absolute prediction error for the original data and the filtered data is shown in Fig. 7. The probability of a smaller prediction error is higher in the HAC-LSTM compared to the other five methods both in the original and filtered data.

Table I shows the prediction errors of different prediction methods. Mean absolute percentage error (MAPE) and root

mean square error (RMSE) are shown. We can see that the prediction errors of all methods after data pre-processing are lower, and the HAC-LSTM has the lowest prediction error.

To sum up, clustering correlated series for prediction can improve prediction accuracy, while uncorrelated series lead to a decrease in prediction accuracy. Filtering out the high-frequency noise components of the spectrum sensing data can improve the correlation between series and further improve the prediction performance.

D. Computational complexity discussion

In this subsection, we compare the computational complexity of the prediction methods used in this paper. The computing complexity of ARIMA method is $O(n_a)$, where $n_a = p + q + d$. The computing complexity of RF method is $O(n_{rt} \times n_{rs} \times n_{rf} \times \log n_{rs})$, where n_{rt} , n_{rs} , and n_{rf} are the number of trees, samples, and features, respectively. The computing complexity of HAC algorithm is $O(n_{hs}^3)$, where n_{hs} is the number of series. The computing complexity of LSTM layer is $O(4n_{li} \times n_{lh} + 4n_{lh}^2 + 3n_{lh} + n_{lh} \times n_{lo})$, where n_{li} , n_{lo} , and n_{lh} are the number of inputs, outputs, and hidden layer units. The computing complexity of CNN layer and ConvLSTM layer are $O((n_{mx} \times n_{my}) \times (n_{fx} \times n_{fy}) \times n_{ci} \times n_{co})$, where $n_{mx} \times n_{my}$ and $n_{fx} \times n_{fy}$ are the size of filter and feature map, and n_{ci} and n_{co} are the number of input and output channels. The computational complexity of the fully connected layer is $O(n_{fci} \times n_{fch} + n_{fch} \times n_{fco})$, where n_{fci} , n_{fch} , and n_{fco} are the number of neurons in input layer, hidden layer, and output layer, respectively. Therefore, the computing complexity of LSTM and HAC-LSTM methods are $O(4n_{li} \times n_{lh} + 4n_{lh}^2 + 3n_{lh} + n_{lh} \times n_{lo} + n_{fci} \times n_{fch} + n_{fch} \times n_{fco})$ and $O(n_{hs}^3 + 4n_{li} \times n_{lh} + 4n_{lh}^2 + 3n_{lh} + n_{lh} \times n_{lo} + n_{fci} \times n_{fch} + n_{fch} \times n_{fco})$, respectively. The computing complexity of CNN-LSTM and ConvLSTM methods are $O((n_{mx} \times n_{my}) \times (n_{fx} \times n_{fy}) \times n_{ci} \times n_{co} + 4n_{li} \times n_{lh} + 4n_{lh}^2 + 3n_{lh} + n_{lh} \times n_{lo} + n_{fci} \times n_{fch} + n_{fch} \times n_{fco})$.

Based on the above analysis, ARIMA method has the lowest computational complexity. In the training process, all deep learning methods have high computational complexity, $M \times S \times O(N)$, where M and S are the number of training rounds and training samples, and $O(N)$ is the computational complexity of deep learning method. If the training process is processed offline and only the prediction part is considered ($M = 1$, $S = 1$), and even reduce some parameters (such as the number of hidden layers and neurons), and the computational complexity can be reduced a lot. The proposed HAC-LSTM in a small-scale wireless sensor network (4 sensors in this paper) has lower computational complexity than CNN-LSTM and ConvLSTM. Moreover, the clustering part is generally performed offline. Thus in the prediction part, the computational complexity of HAC-LSTM method is about the same as LSTM method.

VI. CONCLUSION

In summary, we propose a novel approach of clustering and predicting received power collected by multiple sensors. Based

on CRC's multi-sensors measurement data, we found that the uncorrelated time series may produce mutual interference, and high-frequency noise components in the data series will reduce the prediction accuracy. Based on these observations, our proposed prediction method combines HAC and LSTM to identify and use series with high correlation from multiple sensors. This method improves the prediction accuracy compared with single time series prediction methods. Further, by pre-processing the data through a low-pass filter before clustering and prediction, the performance of each prediction method was improved. The current scale of the test is limited due to the difficulty in obtaining large-scale real sensing data. Further research is acquired in a large-scale wireless sensor network with considerably more sensors. Larger scale network means greater computational complexity, and it is also very important to strike a balance between computational complexity and prediction performance. Suitable clustering methods such as HAC can help to address the scalability issue.

REFERENCES

- [1] L. Ding et al., "Cross-layer routing and dynamic spectrum allocation in cognitive radio Ad Hoc networks," *IEEE Transactions on Vehicular Technology*, 2010, vol. 59, no. 4, pp. 1969–1979.
- [2] J. Guo et al., "ARIMA-based road gradient and vehicle velocity prediction for hybrid electric vehicle energy management," *IEEE Transactions on Vehicular Technology*, 2019, vol. 68, no. 6, pp. 5309–5320.
- [3] C. Ding et al., "Using an arima-garch modeling approach to improve subway short-term ridership forecasting accounting for dynamic volatility," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 4, pp. 1054–1064.
- [4] A. Graves et al., "Speech recognition with deep recurrent neural networks," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, Vancouver, 2013, pp. 6645–6649.
- [5] F. Gers et al., "Learning to forget: continual prediction with LSTM," *Neural Computation*, 2000, vol. 12, no. 10, pp. 2451C–2471.
- [6] Y. Pan et al., "Jointly modeling embedding and translation to bridge video and language," in *The IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4594–4602.
- [7] G. Aceto et al., "Toward effective mobile encrypted traffic classification through deep learning," *Neurocomputing*, 2020, vol. 409, pp. 306–315.
- [8] L. Yu et al., "Spectrum prediction based on taguchi method in deep learning with long short-term memory," *IEEE Access*, 2018, vol. 6, pp. 45923–45933.
- [9] Z. Cao et al., "Improving prediction accuracy in LSTM network model for aircraft testing flight data," in *2018 IEEE International Conference on Smart Cloud*, New York, NY, 2018, pp. 7–12.
- [10] N. Nathani and G. C. Manna, "Predicted call and residual lifetime based channel allocation model for primary user equivalent QoS in cognitive radio cellular network," in *Smart Trends in Information Technology and Computer Communications*, Springer, 2017, vol. 876, pp. 166–278.
- [11] M. Hoyhtya et al., "Measurements and analysis of spectrum occupancy with several bandwidths," in *2013 IEEE International Conference on Communications*, Budapest, 2013, pp. 4682–4686.
- [12] L. Breiman, "Random forests," *Machine learning*, 2001, vol. 45, pp. 5–32.
- [13] J. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, 2002, vol. 38, pp. 367–378.
- [14] Sean J Taylor and Benjamin Letham, "Forecasting at scale," *The American Statistician*, 2018, vol. 72, pp. 37–45.
- [15] B. S. Shawel, D. H. Woldegebreal and S. Pollin, "Convolutional LSTM-based long-term spectrum prediction for dynamic spectrum access," *2019 27th European Signal Processing Conference (EUSIPCO)*, A Coruna, Spain, 2019, pp. 1–5.
- [16] H. Yu et al., "Spatiotemporal recurrent convolutional networks for traffic prediction in transportation networks," *Sensors*, 2017, vol. 17, pp. 1501.
- [17] L. Li et al., "A cloud-based spectrum environment awareness system," *2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications*, Montreal, QC, 2017, pp. 1–6.